PANDIT DEENDAYAL ENERGY UNIVERSITY

(PDEU)



IOT based Server room Monitering System

Project Report

Submitted by

Tanisha Diyora (23BIT188)
Ayushi Maniar (23BIT189)

Submitted to

Dr. Aarti Solanki

PDEU

Declaration

We declare that this report describes our own project work titled “IoT-Based Server Room Monitoring System,” developed as part of our academic curriculum in the Information and Communication Technology (ICT) branch at Pandit Deendayal Energy University (PDEU).

The project focuses on designing a server room monitoring system using an ESP32 microcontroller, MQ-2 gas sensor, DHT11 temperature and humidity sensor, magnetic door sensor, PIR motion sensor, OLED display, red LED, and buzzer. The system is designed to monitor environmental conditions and door activity and provide alerts when predefined conditions are detected.

We confirm that the information presented in this report is based on our project work and that external sources, diagrams, and technical references used have been acknowledged appropriately. Any experiments or hardware tests that have not yet been completed will be marked as pending rather than reported as verified results.

Tanisha Diyora (23BIT188) Ayushi Maniar(23BIT189)

Abstract

Server rooms are essential for storing and processing data and supporting networking infrastructure. These rooms require suitable environmental conditions and controlled physical access to ensure the reliable operation of electronic equipment. Increased temperature, smoke or combustible-gas indications, unexpected movement, and unauthorized door opening may require immediate attention.

The IoT-Based Server Room Monitoring System is designed using an ESP32 microcontroller and multiple sensors to monitor selected environmental and access-related conditions. The MQ-2 sensor detects changes associated with smoke and certain combustible gases, while the DHT11 sensor measures temperature and humidity. A magnetic door sensor monitors door status, and a PIR sensor detects movement within its sensing area.

The ESP32 processes the sensor readings and displays the information on a 0.96-inch OLED display. A red LED and buzzer provide visual and audible alerts when predefined conditions are detected. A relay-controlled DC fan can also be included as an optional feature for temperature-based cooling after proper electrical connections and testing.

The proposed system provides a low-cost educational prototype for demonstrating real-time sensor monitoring and automatic alert generation. It helps students understand the integration of hardware components, embedded programming, and IoT concepts in a practical application. The system focuses on local monitoring; remote notifications and cloud-based monitoring can be added in future versions.

Keywords: Internet of Things, ESP32, server room monitoring, MQ-2 sensor, DHT11, PIR sensor, magnetic door sensor, OLED display, buzzer, embedded systems.

Table of Contents

1. Introduction, Motivation and Problem Statement
2. Objectives, Scope and Proposed System Overview
3. Literature Review and Comparison of Approaches
4. Hardware Components: ESP32 and Sensors
5. System Architecture and Workflow
6. Sensor Readings and Data Processing
7. Software Implementation and Hardware Integration
8. Conclusion and Future Improvements

List of Tables

- Table 1: Comparison of Server Room Monitoring Approaches
- Table 2: Hardware Components and Their Functions
- Table 3: ESP32 Pin Connections
- Table 4: Monitoring Parameters and Alert Conditions
- Table 5: Hardware Testing Plan
- Table 6: Test Results
- Table 7: Limitations and Future Improvements

Abbreviations

Abbreviation	Full form
IoT	Internet of Things
ESP32	ESP32 Microcontroller Platform
GPIO	General-Purpose Input/Output
ADC	Analog-to-Digital Converter
OLED	Organic Light-Emitting Diode
PIR	Passive Infrared
I²C	Inter-Integrated Circuit
DHT	Digital Humidity and Temperature Sensor
LED	Light-Emitting Diode
USB	Universal Serial Bus
VCC	Positive Power Supply
GND	Ground
SDLC	Software Development Life Cycle

1. Introduction, Motivation and Problem Statement

1.1 Background

Server rooms are essential facilities that contain servers, routers, switches, and storage devices. These devices generate heat and require suitable environmental conditions to operate reliably. High temperatures, humidity changes, smoke, and unexpected access can affect equipment safety and performance.Traditional monitoring often depends on manual inspection, which may not identify problems immediately. This project uses an ESP32 and multiple sensors to monitor important server-room conditions and provide alerts when abnormal readings are detected.

1.2 Introduction to IoT

The Internet of Things (IoT) connects physical devices equipped with sensors and communication capabilities to collect and exchange information.An IoT monitoring system typically includes sensors, a microcontroller, communication interfaces, output devices, and a monitoring interface. In this project, the ESP32 processes sensor readings, while the OLED displays information and the red LED and buzzer provide alerts. Remote monitoring would require additional network and software integration.

1.3 Motivation

Server rooms require regular monitoring because abnormal environmental conditions can affect electronic equipment. Manual inspection may not detect changes between checks.The motivation behind this project is to develop a low-cost prototype that combines multiple sensors in one system. It also provides practical experience with ESP32 programming, sensor interfacing, circuit design, and automated monitoring.

1.4 Problem Statement

The aim is to design and implement an ESP32-based server-room monitoring prototype that monitors temperature, humidity, smoke-related sensor responses, door status, and movement. The system provides visual and audible alerts when predefined conditions are detected, offering a simple and economical approach to local monitoring.

1.5 Nature of the System Output

The system displays sensor readings and indicates when configured alert conditions are triggered. An MQ-2 reading above a threshold indicates a sensor response that requires attention but does not confirm an exact gas concentration. Similarly, a PIR sensor detects movement but cannot identify a person.

The prototype is designed for academic demonstration and does not replace certified fire detection, professional access control, or commercial server-room monitoring equipment.

2. Objectives, Scope and Proposed System Overview

2.1 Objectives

The main objective is to develop an IoT-based server room monitoring prototype using an ESP32 and multiple sensors.

The specific objectives are:

- To measure temperature and humidity using the DHT11 sensor.
- To monitor smoke- and certain combustible-gas-related sensor responses using the MQ-2 sensor.
- To detect door contact status using a magnetic door sensor.
- To detect movement using a PIR sensor.
- To display sensor readings on an OLED display.
- To generate a visual alert using a red LED.
- To generate an audible alert using a buzzer.
- To implement sensor reading and alert logic using Arduino IDE.
- To test the operation of the connected hardware components.
- To explore automatic fan control through a relay as an optional feature.

2.2 Scope

The scope of the project includes hardware assembly, sensor interfacing, ESP32 programming, local display of readings, and alert generation.

The system is designed to monitor multiple conditions through a single controller. It can be used to demonstrate how sensor readings are collected and processed and how a microcontroller can control output devices.

The current implementation focuses on local monitoring through the OLED display, red LED, buzzer, and Serial Monitor.

The following features are outside the current scope unless implemented separately:

- Cloud-based data storage
- Mobile application notifications
- Remote monitoring through a web dashboard

- Identification of individuals entering the room
- Certified fire detection
- Professional server-room cooling and access-control systems

2.3 Proposed System Overview

The proposed system consists of three main layers.

Input layer: The MQ-2, DHT11, magnetic door sensor, and PIR sensor collect information about selected environmental and physical conditions.

Processing layer: The ESP32 reads the sensor outputs and evaluates them according to predefined rules and thresholds.

Output layer: The OLED displays the readings, while the red LED and buzzer provide warnings. An optional relay can control a fan if the required hardware is correctly connected.

The controller repeatedly performs the monitoring process, allowing changes in sensor readings to be reflected in the displayed status.

2.1 Objectives

The main objective is to develop an IoT-based server-room monitoring prototype using an ESP32 and multiple sensors.The specific objectives are:

- To measure temperature and humidity using the DHT11 sensor.
- To monitor smoke-related sensor responses using the MQ-2 sensor.
- To detect door status using a magnetic door sensor.
- To detect movement using a PIR sensor.
- To display readings on an OLED display.
- To generate visual and audible alerts using a red LED and buzzer.
- To program the ESP32 using Arduino IDE and test the connected components.
- To explore automatic fan control using an optional relay.

2.2 Scope

The project covers hardware assembly, sensor interfacing, ESP32 programming, displaying readings, and generating alerts. It demonstrates how multiple sensors can be connected to a single controller for local monitoring.The current implementation focuses on the OLED display, red LED, buzzer, and Serial Monitor. Cloud storage, mobile notifications, web

dashboards, person identification, certified fire detection, and professional cooling systems are outside the current scope unless implemented separately.

2.3 Proposed System Overview

The proposed system consists of three main layers:

- Input Layer: The MQ-2, DHT11, magnetic door sensor, and PIR sensor collect environmental and physical information.
- Processing Layer: The ESP32 reads the sensor outputs and checks them against predefined conditions and thresholds.
- Output Layer: The OLED displays readings, while the red LED and buzzer provide alerts. An optional relay can control a fan when suitable hardware is connected.

The ESP32 continuously repeats this process to update sensor readings and system status.

2.4 Proposed System

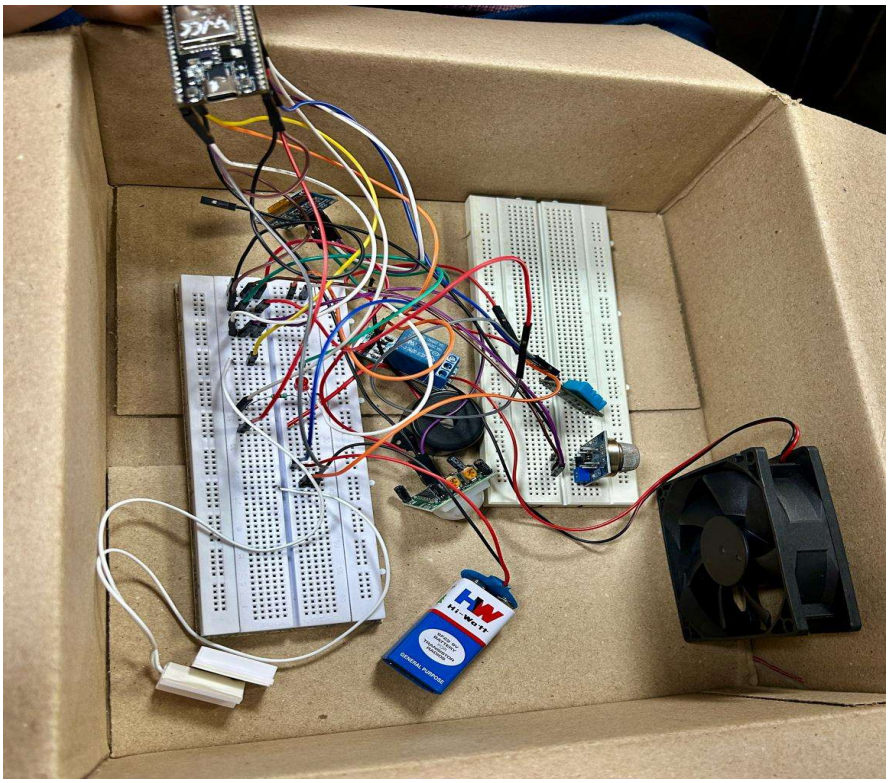


Figure 1: IoT-Based Server Room Monitoring System

3. Literature Review and Comparison of Approaches

3.1 IoT in Server Room Monitoring

IoT technologies can improve monitoring by connecting sensors, controllers, and software systems. When communication features are implemented, sensor information can also be sent to remote dashboards or storage services.

An IoT-based monitoring system can help users observe environmental changes and identify conditions that require attention. However, the usefulness of the system depends on the accuracy of the sensors, reliability of the communication system, and correctness of the software logic.

In the current project, the ESP32 processes the inputs locally. Remote monitoring is a possible extension rather than an automatically available feature.

3.2 Comparison of Monitoring Approaches

Table 1: Comparison of Server Room Monitoring Approaches

Approach	Advantages	Limitations
Manual inspection	Simple and inexpensive	Not continuous; depends on human observation
Standalone sensors	Easy to use for individual parameters	Separate devices may be required
Commercial systems	Advanced monitoring and reporting	Can involve higher costs
ESP32-based prototype	Flexible, low-cost and customizable	Requires programming, calibration and testing

3.3Positioning of the Proposed Project

The proposed project focuses on an affordable educational implementation. It combines environmental monitoring, door-status monitoring, motion detection, a display, and local alerts.

The main purpose is to demonstrate how multiple sensors can be interfaced with a microcontroller and used to create a unified monitoring system.

The project does not claim to provide the same accuracy or reliability as professionally installed server-room monitoring equipment.

4. Hardware Components: ESP32 and Sensors

4.1 Hardware Requirements

The project uses an ESP32 development board as the main controller. The sensors provide the input information, and the display and alert devices communicate the system's status to the user.

Table 2: Hardware Components and Their Functions

Component	Function
ESP32 development board	Processes sensor inputs and controls outputs
MQ-2 gas sensor	Detects changes associated with smoke and certain combustible gases
DHT11 sensor	Measures temperature and relative humidity
Magnetic door sensor	Detects changes in door contact state
PIR sensor	Detects movement in its sensing area
0.96-inch OLED display	Displays readings and system status
Red LED	Provides a visual warning
Buzzer	Provides an audible warning
Relay module (optional)	Switches a separate electrical load
DC fan (optional)	Provides airflow when correctly powered
Breadboard	Supports temporary circuit assembly
Jumper wires	Connect the modules and controller
USB cable and suitable power supplies	Provide power and allow programming

4.2 ESP32 Development Board

The ESP32 is the main controller. It reads sensor data, processes alert conditions, and controls output devices. It supports GPIO, ADC, I²C, Wi-Fi, and Bluetooth.

4.3 MQ-2 Gas Sensor

The MQ-2 detects smoke and certain combustible gases. The ESP32 reads its analog output and compares it with a set threshold to trigger alerts. The sensor requires warm-up and calibration, and its output voltage must be safe for the ESP32.

4.4 DHT11 Temperature and Humidity Sensor

The DHT11 measures temperature and humidity. The ESP32 receives the readings through GPIO 4 and uses them to display environmental conditions and detect high-temperature alerts.

4.5 Magnetic Door Sensor

The magnetic door sensor detects whether a door is open or closed using a magnet and switch. It connects to GPIO 27, with the input logic verified according to the actual wiring.

4.6 PIR Motion Sensor

The PIR sensor detects movement by sensing changes in infrared energy. Its output is read by the ESP32 through GPIO 26 to identify motion and trigger alerts.

4.7 OLED Display

The 0.96-inch OLED displays sensor readings, door status, motion status, and alerts. It uses I²C communication, with SDA on GPIO 21 and SCL on GPIO 22. The common display address is 0x3C.

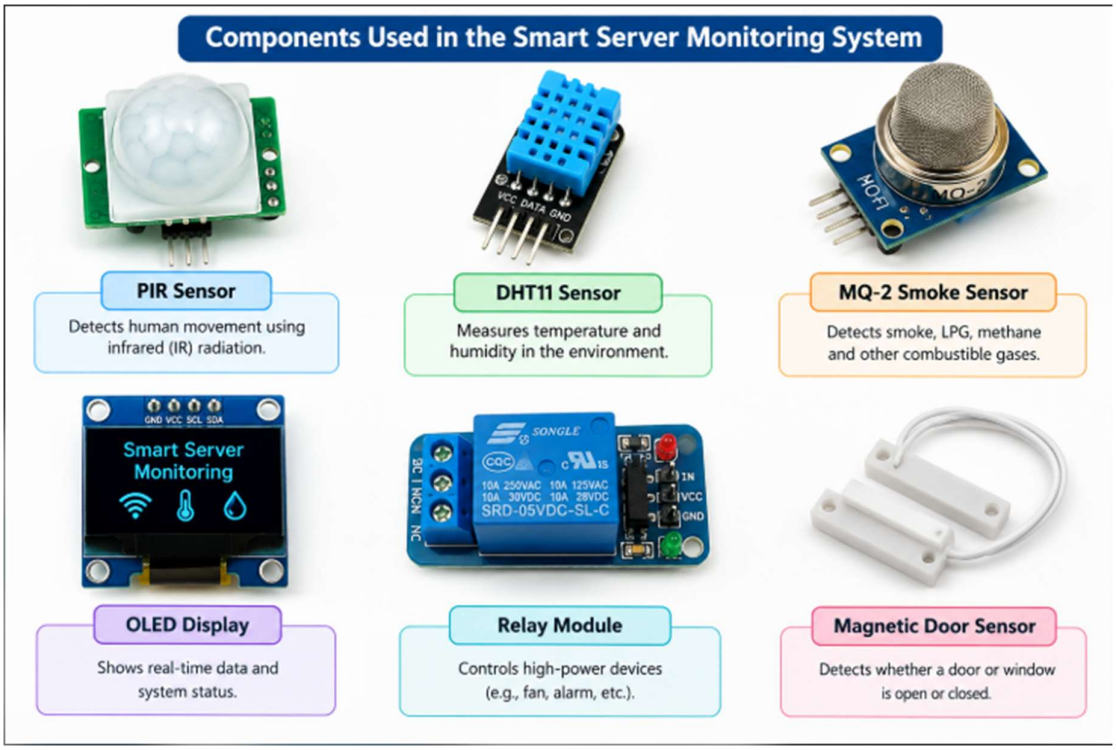
4.8 Red LED and Buzzer

The red LED provides a visual warning, while the buzzer provides an audible alert when a condition is detected. The LED uses GPIO 19 with a current-limiting resistor. The buzzer must be connected according to its electrical requirements.

4.9 Optional Relay and Fan

A relay can control a DC fan when a programmed condition, such as high temperature, occurs. The relay and fan require suitable power supplies and ratings. If not installed, they should be considered future enhancements.

Image of Components used in the system



5. System Architecture and Workflow

5.1 System Architecture

The proposed system follows an input-processing-output architecture. The sensors collect information, the ESP32 processes the information, and the output devices communicate the resulting status.

The input layer consists of the MQ-2, DHT11, magnetic door sensor, and PIR sensor. The processing layer consists of the ESP32 and its firmware. The output layer consists of the OLED display, red LED, and buzzer, with an optional relay and fan.

5.2 Hardware Connections

The following table provides the proposed pin configuration for the ESP32 setup discussed in the project.

Table 3: Pin Connections of the Proposed System

Component	Component pin	ESP32 connection
MQ-2	AO	GPIO 34 through a suitable voltage divider
MQ-2	VCC	Rated supply, typically 5V for the module
MQ-2	GND	GND

DHT11	DATA	GPIO 4
DHT11	VCC	3.3V, if supported by the module
DHT11	GND	GND
Door sensor	Wire 1	GPIO 27
Door sensor	Wire 2	GND
PIR sensor	OUT	GPIO 26
PIR sensor	VCC	Supply according to module rating
PIR sensor	GND	GND
OLED	SDA	GPIO 21
OLED	SCL	GPIO 22
OLED	VCC	Suitable supply according to module rating
OLED	GND	GND
Red LED	Anode	GPIO 19 through a resistor
Red LED	Cathode	GND
Buzzer	Positive	GPIO 18 through a suitable driver if required
Buzzer	Negative	GND
Optional relay	IN	GPIO 25
Optional relay	VCC	Regulated 5V supply, if required
Optional relay	GND	Common ground, as required by the circuit

The table must be checked against the actual components before power is applied. ESP32 GPIO pins are not 5V tolerant, and the MQ-2 analog output may require voltage reduction.

5.3 System Workflow

The system operates through the following sequence:

1. The ESP32 is powered on.
2. The firmware initializes the sensors, OLED, and output pins.
3. The MQ-2 sensor is allowed to warm up appropriately.

4. The ESP32 reads the analog gas-sensor value.
5. The DHT11 provides temperature and humidity readings.
6. The door sensor and PIR sensor provide their digital states.
7. The firmware evaluates the readings and programmed thresholds.
8. The red LED and buzzer are activated if an alert condition is detected.
9. The OLED displays the current readings and status.
10. The monitoring cycle repeats continuously.

6. Sensor Readings and Data Processing

The ESP32 receives information from sensors in different formats. Some components provide analog readings, while others provide digital signals or formatted measurements.

The firmware processes these inputs and converts them into information that can be displayed and used for alert generation.

The DHT11 measures temperature in degrees Celsius and relative humidity as a percentage, and the program checks that the readings are valid before displaying them. A temperature threshold of 35°C is used to trigger a high-temperature alert. The magnetic door sensor indicates whether the door is open or closed, while the PIR sensor detects movement. During each monitoring cycle, the ESP32 collects sensor readings, validates the temperature and humidity data, checks the digital sensor states, and compares the MQ-2 gas reading and temperature against their configured thresholds. Based on these results, it updates the OLED display and activates the appropriate alerts.

7. Software Implementation and Hardware Integration

7.1 Software Requirements

The firmware is developed using Arduino IDE. The IDE allows the user to write the program, compile it, and upload it to the ESP32 development board.

The following software components are required:

- Arduino IDE for programming and uploading.
- ESP32 board support package for compiling the program.
- Adafruit GFX Library for graphics and text functions.
- Adafruit SSD1306 Library for controlling the OLED display.
- DHT sensor library by Adafruit for reading the DHT11.
- Adafruit Unified Sensor library as a dependency where required.

- The built-in Wire.h library for I²C communication.

The installed libraries and board package must be compatible with the selected ESP32 board.

7.2 Program Used

The firmware is organized into initialization, sensor reading, condition evaluation, output control, and display updating.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

// ================ OLED ================
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define OLED_ADDRESS 0x3C

Adafruit_SSD1306 display(
  SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET
);

// ================ ESP32 PINS ================
const int MQ2_PIN = 34;
const int DHT_PIN = 4;
const int DOOR_PIN = 27;
const int PIR_PIN = 26;
const int BUZZER_PIN = 18;
const int RED_LED_PIN = 19;
const int RELAY_PIN = 25;

// ================ SENSORS ================
#define DHT_TYPE DHT11
DHT dht(DHT_PIN, DHT_TYPE);

// Demonstration thresholds only; calibrate before use.
const int GAS_THRESHOLD = 1800;
const float TEMP_THRESHOLD = 35.0;

```

```
// Change these after testing your hardware.
const bool DOOR_OPEN_IS_HIGH = true;
const bool BUZZER_ACTIVE_HIGH = true;
const bool RELAY_ACTIVE_LOW = true;

// ================ SETUP ================
void setup() {
  Serial.begin(115200);

  pinMode(DOOR_PIN, INPUT_PULLUP);
  pinMode(PIR_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(RED_LED_PIN, OUTPUT);
  pinMode(RELAY_PIN, OUTPUT);

  digitalWrite(BUZZER_PIN, LOW);
  digitalWrite(RED_LED_PIN, LOW);

  // Relay OFF at startup
  digitalWrite(RELAY_PIN, RELAY_ACTIVE_LOW ? HIGH : LOW);

  // OLED I2C pins
  Wire.begin(21, 22);

  // Start DHT11
  dht.begin();

  // Start OLED
  if (!display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDRESS)) {
    Serial.println("OLED not detected. Check wiring/address.");
  } else {
    display.clearDisplay();
```

```
    display.setTextColor(SSD1306_WHITE);
    display.setTextSize(1);
    display.setCursor(0, 0);
    display.println("SERVER ROOM");
    display.println("MONITORING SYSTEM");
    display.println("Initializing...");
    display.display();
  }

  Serial.println("Server Room Monitoring Started");
  delay(3000);
}

// ================ MAIN LOOP ================
void loop() {
  // Read all sensors
  int gasValue = analogRead(MQ2_PIN);
  int doorValue = digitalRead(DOOR_PIN);
  int pirValue = digitalRead(PIR_PIN);

  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();

  // Interpret readings
  bool doorOpen = DOOR_OPEN_IS_HIGH
                  ? (doorValue == HIGH)
                  : (doorValue == LOW);

  bool motionDetected = (pirValue == HIGH);
  bool gasAlarm = (gasValue >= GAS_THRESHOLD);

  bool tempValid = !isnan(temperature);
```

```
131   // ================= SERIAL MONITOR =================
132   Serial.println("\n--- SERVER ROOM STATUS ---");
133
134   Serial.print("MQ-2: ");
135   Serial.println(gasValue);
136
137   Serial.print("Door: ");
138   Serial.println(doorOpen ? "OPEN" : "CLOSED");
139
140   Serial.print("Motion: ");
141   Serial.println(motionDetected ? "DETECTED" : "NONE");
142
143   Serial.print("Temperature: ");
144   if (tempValid) {
145     Serial.print(temperature, 1);
146     Serial.println(" C");
147   } else {
148     Serial.println("DHT11 ERROR");
149   }
150
151   Serial.print("Humidity: ");
152   if (humidityValid) {
153     Serial.print(humidity, 1);
154     Serial.println(" %");
155   } else {
156     Serial.println("DHT11 ERROR");
157   }
158
159   Serial.print("Fan: ");
160   Serial.println(fanOn ? "ON" : "OFF");
```

```
  // ================ OLED DISPLAY ================
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);

  display.println("SERVER ROOM MONITOR");
  display.println("--------------------");

  display.print("Gas: ");
  display.println(gasValue);

  display.print("Door: ");
  display.println(doorOpen ? "OPEN" : "CLOSED");

  display.print("Motion: ");
  display.println(motionDetected ? "YES" : "NO");

  display.print("Temp: ");
  if (tempValid) {
    display.print(temperature, 1);
    display.println(" C");
  } else {
    display.println("ERROR");
  }

  display.print("Humidity: ");
  if (humidityValid) {
    display.print(humidity, 1);
    display.println(" %");
  } else {
    display.println("ERROR");
```

```
  display.print("Humidity: ");
  if (humidityValid) {
    display.print(humidity, 1);
    display.println(" %");
  } else {
    display.println("ERROR");
  }

  display.print("Fan: ");
  display.println(fanOn ? "ON" : "OFF");

  display.print("Status: ");
  display.println(alarm ? "ALERT!" : "NORMAL");

  display.display();

  delay(1000);
}
```

7.3 Hardware Integration

After the firmware is uploaded, the system is tested with the connected sensors.

The OLED is checked first to confirm that the display initializes correctly. The DHT11 readings are then observed, followed by individual tests of the door sensor, PIR sensor, MQ-2, red LED, and buzzer.

Each component is verified separately before the complete monitoring system is tested.

If the relay and fan are installed, their switching behaviour is verified independently before integrating the temperature-control rule.

8. Testing, Evaluation, Results and Limitations

8.1 Testing Methodology

Testing is performed to verify that each sensor communicates correctly with the ESP32 and that the output devices respond to the programmed conditions.

The testing process begins with a visual inspection of the hardware connections. Power supplies, sensor polarity, signal wiring, and common ground connections are checked before powering the system.

After uploading the firmware, the Serial Monitor is used to observe the readings. Each component is tested individually, followed by an integrated test of the monitoring system.

8.2 Testing Plan

Table 5: Hardware Testing Plan

Test case	Procedure	Expected observation
TC1: OLED	Power on the system	Display initializes and shows information
TC2: DHT11	Read temperature and humidity	Valid readings are displayed
TC3: Door sensor	Open and close the door	Door status changes
TC4: PIR sensor	Move within the sensing area	Motion status changes
TC5: MQ-2	Observe sensor readings after warm-up	Analog reading is available
TC6: Red LED	Trigger an alert condition	LED activates
TC7: Buzzer	Trigger an alert condition	Audible warning is produced
TC8: Overall status	Evaluate normal and alert conditions	OLED status changes appropriately

TC9: Relay and fan	Test the optional switching circuit	Fan follows the verified control rule

8.4 Results

Table 6: Test Results

Parameter	Actual observation
MQ-2 reading	
Temperature	°C
Humidity	%
Door sensor	
PIR sensor	
OLED display	
Red LED	
Buzzer	+
Relay and fan	

8.5 Limitations

The current system has several limitations:

1. Sensor accuracy: The DHT11 is a basic sensor and may not provide the accuracy needed for professional environmental control.
2. MQ-2 selectivity: The MQ-2 responds to multiple gases and environmental factors and cannot independently identify a specific gas.
3. PIR detection: The PIR sensor detects movement but cannot identify a person.
4. Door monitoring: The magnetic door sensor reports contact state but does not identify who opened the door.
5. Threshold selection: Incorrect thresholds can produce false alerts or missed conditions.

9. Conclusion and Future Improvements

9.1 Conclusion

The IoT-Based Server Room Monitoring System uses an ESP32 and multiple sensors to monitor temperature, humidity, smoke-related gas levels, door status, and motion. Sensor readings are processed and displayed on an OLED, while the red LED and buzzer provide alerts when predefined conditions are detected. An optional relay and fan can demonstrate automatic cooling. This project provides practical experience in sensor interfacing, embedded programming, and IoT concepts. The prototype is intended for educational purposes and is not a replacement for professional monitoring or certified safety systems.

9.2 Future Improvements

1. Wi-Fi Monitoring: Send sensor readings to a cloud platform for remote access.
2. Web Dashboard: Display real-time readings and historical charts.
3. Mobile Notifications: Notify users when alert conditions occur.
4. Data Logging: Store sensor readings for future analysis.
5. Improved Sensors: Use more accurate sensors and certified smoke detectors.
6. Automatic Cooling: Integrate a suitable relay and fan for temperature-based cooling.